gRPC, Protocol Buffers, HTTP/2 and HTTP/3

Contract-first calls, streaming, and the transport underneath them

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Spring 2027

TODAY

What you should leave with

Design the contract

Write a .proto file and generate Go and Dart stubs with protoc, from one source of truth.

Call all four shapes

Unary, server streaming, client streaming and bidirectional, and know which one a screen actually needs.

Reason about the wire

Why protobuf is smaller than JSON, how a schema evolves safely, and what HTTP/2 buys for free.

Survive the network

Deadlines, cancellation and the radio tail: the habits that keep a mobile client polite.

OVERVIEW

What gRPC actually is

Contract first

gRPC is a Remote Procedure Call (RPC) framework: one .proto file is the source of truth for every client and the server.

Code generation

protoc turns messages and services into typed classes, in every language you target, from the same file.

Binary and HTTP/2

Compact binary frames, multiplexed on one warm connection instead of text on many short-lived ones.

Streaming built in

Four call shapes are part of the contract itself, not a library bolted on afterward.

PART 1 · CONTRACTS

Designing the call before writing code

RPC versus REST, and the .proto contract

RPC models verbs, REST models nouns

REST (Representational State Transfer) models nouns. You name resources and use a few fixed verbs on them, a good fit for a public API that strangers discover from a URL.

RPC models verbs. You name the operation your app needs, such as GetDashboard, and call it like a local function. Client and server agree on a list of procedures, not a URL space.

On a phone the difference is round trips. A resource-shaped screen often needs three or four sequential requests, and each one costs real time on cellular before any data moves.

```
# REST: four requests
GET /users/42
GET /users/42/devices
GET /devices/9/readings
GET /devices/9/alerts

# RPC: one call
GetDashboard(user_id: 42)
  -> Dashboard
```

Design the call around the screen. Not around your database tables. The round trip is the expensive part.

Choosing between REST and gRPC

QUESTION	FAVOR REST	FAVOR GRPC
Who calls it?	Strangers, from a URL	You control both ends
Needs streaming?	Rarely	Built into the contract
Browser calls it?	Yes, plain HTTP	Only via gRPC-Web
Payload format?	JSON, readable	Binary, needs tooling
Latency budget?	One request is fine	Every round trip counts

Neither one is modern. gRPC wins where you own both ends of the call. REST wins where you do not.

Contract first: the .proto file

```
syntax = "proto3";
package telemetry.v1;
message Reading {
  string device_id        = 1;
  double temp_c           = 2;
  int64  taken_at_unix_ms = 3;
}
service Telemetry {
  rpc SubmitReading(Reading) returns
    (SubmitReadingResponse);
  rpc WatchReadings(WatchReadingsRequest)
    returns (stream Reading);
}
```

One file, three teams.
Backend, Android and iOS build from it in parallel. Field numbers are the wire format: names never travel.
Enums need a zero: an absent enum decodes as 0, so 0 must mean UNSPECIFIED.

Generating stubs with protoc

Run protoc once per language, pointed at the same .proto file. Commit the generated code and regenerate on change, or run protoc in CI. Pick one and write it down.

```
# Go: two plugins, messages and service
protoc -Iproto \
  --go_out=gen      --go_opt=paths=source_relative \
  --go-grpc_out=gen --go-grpc_opt=paths=source_relative \
  proto/telemetry/v1/telemetry.proto

# Dart: the plugin is a pub global executable
dart pub global activate protoc_plugin
protoc -Iproto --dart_out=grpc:lib/gen \
  proto/telemetry/v1/telemetry.proto
```

What the generator produces

Typed message classes. Reading, SubmitReadingResponse and their siblings, as real classes with typed getters, equality and a binary codec already written.

A client stub. TelemetryClient in Dart: the object your app calls as if it were a local class.

A server skeleton. UnimplementedTelemetryServer in Go: embed it, then implement one method at a time.

Generated code is build output. Never edit it by hand. The pubspec needs the protobuf and grpc packages to compile the result.

The compiler writes the parsing code, which is the code you would otherwise get subtly wrong.

PART 2 · IMPLEMENTING

A Go server and a generated Dart client

Unary calls, then a stream of sensor readings

A Go server: the unary call

```
func (s *server) SubmitReading(ctx
    context.Context,
    req *pb.Reading) (*pb.SubmitReadingResponse,
    error) {
    id, err := s.store.Save(ctx, req)
    if err != nil {
        return nil, status.Error(codes.Internal,
    "save failed")
    }
    return &pb.SubmitReadingResponse{StoredId:
    id}, nil
}
```

ctx is the deadline.
Canceled when the phone gives up; the query stops here too.
Errors are codes, not strings: codes.InvalidArgument, Internal, and friends.

Errors are codes, not strings

CODE	MEANING
InvalidArgument	The client sent something the server rejects outright
NotFound	No resource matches the request
Unauthenticated	No valid credentials were presented
Unavailable	The server is temporarily down, safe to retry
DeadlineExceeded	The call's deadline passed before a response arrived

The client branches on `e.code`, not on a parsed string. That is the whole point of a status code.

A Dart client: the unary call

```dart
final channel =
    ClientChannel('api.example.com', port: 443,
    options: const ChannelOptions(
        credentials:
    ChannelCredentials.secure()));
final client = TelemetryClient(channel);
final res = await client.submitReading(
  Reading()..deviceId = id..tempC = 21.5,
  options: CallOptions(timeout: const
    Duration(seconds: 2)),
);
debugPrint(res.storedId);
```

It looks like a local call. That is the point of RPC, and the danger: it is still the network.

Always set a deadline. Without one, a stalled call holds a UI state forever.

Server streaming: the Go side

```
func (s *server) WatchReadings(
    req *pb.WatchReadingsRequest,
    stream pb.Telemetry_WatchReadingsServer,
) error {
    ctx := stream.Context()
    for r := range s.store.Sub(ctx,
     req.DeviceId) {
        if err := stream.Send(r); err != nil {
            return err
        }
    }
    return nil
}
```

One method, many responses. WatchReadings is declared once, as returns (stream Reading). The handler loops and sends until the client goes away.
ctx is still the deadline or the client's cancel. stream.Send fails once nobody is listening, and returning ends the stream cleanly.

Server streaming: the Dart side

```
final call = client.watchReadings(
  WatchReadingsRequest()..deviceId = id,
  options: CallOptions(timeout: const
    Duration(seconds: 5)));
final sub = call.listen(_onReading,
    onError: _onError);
@override
void dispose() {
  sub.cancel(); channel.shutdown();
  super.dispose();
}
```

Cancel in dispose().
An open stream keeps the radio and the server busy for a screen the user already left.
The deadline still applies: a stream silent that long fails too.

Listen, cancel, and set a deadline: the three habits Lab 6 grades directly.

Client streaming: draining an offline queue

Many requests, one response: the shape of an outbox drain from Lecture 9. Queue rows offline, then stream them upstream for one acknowledgement.

```
// service Telemetry adds:
// rpc UploadReadings(stream Reading)
    returns (Ack);
Stream<Reading> _drain() async* {
  for (final r in await outbox.rows())
    {
    yield r.toReading();
  }
}
final sum = await
    client.uploadReadings(_drain());
debugPrint('stored: ${sum.accepted}');
```

PART 3 · THE WIRE

What actually travels, and how it changes

Protobuf’s size on the wire, and evolving a schema safely

Why protobuf is smaller, and by how much

It is binary. No quotes, commas or braces, and numbers are not rendered as text.

Fields are numbered, not named. A tag byte packs the field number and its wire type, so a field name never travels.

Integers are varints. Seven bits of value per byte, so small numbers cost one byte instead of four.

Defaults are free. Nothing is sent for a field left at its default value.

```
# JSON: 38 bytes
{"device_id":"esp32-a1","temp_c":21.5}
# protobuf: 19 bytes
0A 08 65 73 70 33 32 2D 61 31
11 00 00 00 00 00 80 35 40
# varint: 300 -> AC 02 (2 bytes)
```

Quote the mechanism, not a multiplier. Protobuf is smaller, often by a large factor, but the ratio depends on your schema and data. Measure yours.

Schema evolution without breaking old apps

```
message Reading {
  string device_id        = 1;
  double temp_c           = 2;
  int64  taken_at_unix_ms = 3;
  double humidity         = 5;  // new
  reserved 4;                   // retired
}
```

Forward: old app, new server.
The parser skips unknown tag 5 by its wire type. It does not crash.
Backward: a missing field decodes to its default. Never renumber, reuse, or retype a field.

Users do not update. A year-old build is still on the network, and the wire format is the compatibility contract.

Four call shapes, not one

SHAPE	SIGNATURE	WHEN
Unary	`rpc Get(Req) returns (Res)`	One request, one response: most screens
Server streaming	`returns (stream Reading)`	One request, many responses: live feeds
Client streaming	`(stream Reading) returns (Ack)`	Many requests, one response: draining a queue
Bidirectional	`(stream In) returns (stream Out)`	Both directions at once: chat, presence

The shape is part of the contract. Changing a method from unary to streaming changes the generated signature on every client.

PART 4 · TRANSPORT

HTTP/2, HTTP/3, and what a phone pays for

One warm connection, and what happens when the network changes

TRANSPORT

HTTP/2: one connection, many streams

HTTP/2 multiplexes calls over one connection. HTTP/1.1 allowed one request at a time; browsers hid that with six connections. gRPC rides on this: one call per stream.

What it does not fix: TCP delivers in order. One lost packet stalls every stream on that connection until it is retransmitted.

One TCP + TLS connection. Handshake once, reuse it for every call.

HPACK, HTTP/2's header compression. A repeated header costs a table index, not the bytes.

Streams are independent. A slow call does not block a fast one.

One connection, kept warm. That is most of gRPC's mobile advantage, before a single byte of protobuf is counted.

HTTP/3 and QUIC: the mobile case

QUIC moves onto UDP and implements streams itself. A lost packet now stalls only that stream, so HTTP/2's head-of-line blocking goes away.

Transport Layer Security (TLS) 1.3 is folded into the handshake. A new connection is one round trip; a resumed one can be zero.

Connection migration matters here. A QUIC connection has an id, not an IP and port, so it survives Wi-Fi to 5G.

Where gRPC stands. The wire protocol is specified over HTTP/2.

In practice. HTTP/3 usually reaches you at the edge: a load balancer speaks it to the device, HTTP/2 to your service.

Treat it as deployment, not a code change.

Handshakes and reconnects are the mobile cost. QUIC addresses both, which is why it belongs in the same argument as the radio tail.

NUMBERS

One connection, four shapes, zero round trips

1

TCP connection, reused for every call

4

call shapes defined by the .proto, not by you

0-RTT

round-trip time on a resumed QUIC connection

Deadlines, cancellation, and the radio tail

A deadline is not a client-side timeout. It travels with the call and propagates through every service behind it. When it expires, work stops everywhere.

The radio has a tail. It stays high-power for seconds after a transmission. One burst allows it to sleep; several chatty requests keep it awake.

iOS caveat

A suspended app gets no CPU and its sockets close, so a long-lived stream does not survive backgrounding. Reconnect on resume; use a silent push or a background task to be woken.

Batch, set a deadline, cancel on dispose. Three habits that show up as battery life in a review.

PART 5 · REACHING EVERYWHERE

The browser, and the project

gRPC-Web, Connect, and where Lab 6 picks this up

gRPC-Web and Connect: the browser problem

A browser cannot speak gRPC. fetch and XHR give no control over HTTP/2 frames and no way to read trailers, where gRPC puts the call's status.

gRPC-Web is a different wire encoding, over HTTP/1.1, for unary and server streaming only. It needs a translating proxy.

Connect is the pragmatic successor: one Go handler serves all three protocols on the same route.

```
mux := http.NewServeMux()
path, h := connect.NewHandler(
    &server{})
mux.Handle(path, h)
http.ListenAndServe(":8080", mux)
```

A Flutter web build cannot open a raw gRPC connection.

The boundary has a shape here too: what the browser sandbox will let you send is part of your architecture.

PROJECT

Where this lands in Lab 6 and the project

Lab 6

One .proto with a unary call and a server-streaming call, a Go server, and a generated Dart client. The stream updates live, canceling stops it, and a two-second deadline fails a call against a server that sleeps three seconds.

The project

Requirement four accepts gRPC as one boundary choice: a Go server with a generated Dart client, alongside FFI or a platform channel from Lecture 12.

Everything on this slide, you now have the vocabulary for. The exact commands and acceptance criteria are in the Lab 6 handout.

RECAP

Three things to remember

1. A .proto is a contract, not a suggestion. Field numbers are the wire format; never renumber or reuse one.
2. The shape of a call is part of the contract. Unary, server streaming, client streaming and bidirectional are four different generated signatures.
3. Deadlines and cancellation make gRPC polite on a phone. Set one, cancel in dispose, and mind the radio tail.

FURTHER READING

grpc.io/docs · protobuf.dev/encoding · RFC 9114 (HTTP/3)

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel